Development Operating Model

> A reference description of how the **Rent-A-Vacation** application is built, shipped, governed, and kept coherent across long-running AI-assisted sessions. Use this as a sharable artifact for stakeholders, and as a starter template for new applications you spin up.

Application: Rent-A-Vacation (RAV) marketplace platform **Primary repo:** `rent-a-vacation/rav-website` **Audience:** Founders, engineers, advisors, and future you starting a new project **Last reviewed:** May 2026

---

1. Why this document exists

A solo / small-team operator working with AI assistants needs three things to ship credibly:

1. **A predictable workflow** — branches, deploys, and reviews that behave the same way every time.
2. **A coherent memory** — docs, decisions, and session handoffs that survive long gaps.
3. **A reproducible quality bar** — automated tests, lint, doc audits, and observability that catch drift.

This doc inventories what RAV uses today for each of those, *and* records the alternatives that were considered and rejected so the trade-offs are visible.

---

2. Source-control & branching model

2.1 Branch topology

```
feature/* (optional)
    |
    └─▶ dev (working branch, auto-preview deploy)
              |
              └─▶ main (production, protected)
```

Branch	Purpose	Deploys to	Protection
`feature/*`	Larger changes worth isolating	Vercel preview per branch	None

`dev`	Default working branch — small fixes, mid-size features	Vercel **preview** (`rentavacation-git-dev-…vercel.app`) wired to **Supabase DEV** project (`oukbxqnlxnkainnligfz`)	Required for CI to pass
`main`	Production	Vercel **production** (`rentavacation.vercel.app` + `rent-a-vacation.com`) wired to **Supabase PROD** (`xzfllqndrlmhclqfybew`)	PR + 1 review + CI green; no direct push

2.2 Rules

- **Never push directly to `main`.** Always PR from `dev` (or a feature branch).
- Local `.env.local` points to **Supabase DEV** so local dev never mutates production.
- Feature branches are *optional* — small changes go straight to `dev`.

2.3 Commit message format

Conventional-commits style, but enforced by **convention**, not commitlint (deliberately — see §13 *Tooling decisions*):

```
type(scope): description

feat(auth): add user approval system
fix(voice): correct quota display for tier-based limits
docs(hub): update PROJECT-HUB after session 17
test(booking): add payment flow integration tests
chore(deps): update supabase client to v2.x
```

---

3. Deployment pipeline

The deploy story is intentionally boring:

Stage	Trigger	Mechanism	Target
Preview	Push to any branch	**Vercel GitHub App** (no workflow file needed)	`*-git-<branch>.vercel.app`
Production	Merge to `main`	Vercel GitHub App	`rent-a-vacation.com`
DB migrations	Manual `supabase db push` after PR review	`supabase` CLI from local	Supabase DEV → PROD

No `VERCEL_TOKEN` or deploy workflow exists in the repo — Vercel handles it via the installed GitHub app. The only Vercel artifact in source is `vercel.json`, which configures security headers (CSP,

HSTS, X-Frame-Options, Permissions-Policy) and SPA rewrites.

Considered & rejected:

- *Manual `vercel deploy` from CI* — adds a token to manage and a workflow to maintain for zero gain over the GitHub app.
- *Separate staging Supabase* — DEV doubles as staging; the cost of a third project wasn't justified.

---

4. Continuous integration

All CI runs on **GitHub Actions**, files live in `.github/workflows/`:

Workflow	Trigger	What it does
`ci.yml`	Push to `main`, PR → `main`	Lint (ESLint) → typecheck (`tsc`) → unit/integration (Vitest) → E2E (Playwright) → visual regression (Percy) → Lighthouse CI. Node 20.
`docs-audit.yml`	Push to `dev`/`main`, PR → `main`	Runs `scripts/docs-audit.ts` and `scripts/docs-sync-check.ts`. Comments on PRs when docs are stale; uploads audit artifacts; **fails the build if Tier-1 docs drift** (see §8).
`daily-summary.yml`	Cron 04:30 UTC daily + manual	Pulls GitHub stats (issues opened/closed/merged, code activity, blockers), renders an HTML status report, sends via Resend.
`issue-notifications.yml`	Issue events + comments	Sends HTML email via Resend to four team addresses on issue create/assign/close/comment.
`sync-support-docs.yml`	Push to `main` touching `docs/support/**`, manual	Posts modified support markdown to the Supabase edge function `ingest-support-docs`, which upserts the `support_docs` table that powers the in-app help.

Considered & rejected:

- *CircleCI / Buildkite* — GitHub Actions is co-located with the repo and free for this scale.
- *A separate "deploy" workflow* — Vercel does it.

---

5. Local quality gates

5.1 Husky + lint-staged

Single pre-commit hook at `.husky/pre-commit`:

```
bash scripts/docs-stamp.sh
npx lint-staged
```

- `scripts/docs-stamp.sh` auto-stamps `last_updated` and `change_ref` frontmatter on any staged markdown so the docs audit (§8) stays accurate without manual bookkeeping.
- `lint-staged` runs `eslint --fix` and `vitest related --run` on every staged `.ts`/`.tsx`. This means **only tests touching the changed files run on commit** — fast feedback, but the full suite still runs in CI.

5.2 What is *not* enforced locally (deliberately)

- **No Prettier.** ESLint formatting rules are the source of truth — one tool, fewer fights.
- **No commitlint.** Convention is documented in `CLAUDE.md`; enforcement felt heavier than the value at this team size.
- **No `pre-push` hook.** The CI on PR catches what matters, and a slow push hook breaks flow.

---

6. Issue tracking & project management

6.1 GitHub is the single source of truth

Everything that *might be done* lives as a **GitHub Issue** in `rent-a-vacation/rav-website`. Priorities are expressed via **Milestones** (e.g., `Launch Readiness`, `Phase 20: Accounting & Tax`) and **Labels**.

Label	Use
`bug`	Something broken
`enhancement`	New feature / improvement
`idea`	Unvalidated concept (no milestone)
`docs`	Documentation only
`refactor`	Code quality, no behavior change
`marketplace`	Bidding, listings, booking engine
`platform`	Auth, payments, infra, admin
`experience`	UI, voice, mobile, discovery
`pre-launch`	Required before public launch

`post-launch`	Can wait until after launch
`blocked`	Waiting on something
`needs-decision`	Requires human decision

6.2 Lifecycle

```
new idea  ─▶ issue (label: idea, no milestone)
          ─▶ milestone added           (Ready)
          ─▶ label: in-progress         (Active)
          ─▶ closed with summary        (Done)
```

Closing comments follow a fixed format so the daily summary email can extract them:

```
Completed: <plain-language sentence the owner/renter would understand>

Technical: <migration IDs, test counts, deploy notes — optional>
```

The `Completed:` line surfaces in the **Key Deliverables** section of the daily Resend digest.

6.3 Why GitHub-native, not Linear/Jira?

- The AI assistant (Claude Code) has a first-class **GitHub MCP** server (§7.3); a Linear MCP would add another auth surface for marginal gain.
- Issues, PRs, milestones, and code review live in one URL space — fewer context switches.

---

7. AI-assisted development with Claude Code

The day-to-day work happens inside **Claude Code** (Anthropic's CLI). The harness is configured per-repo through three files and a small set of MCP servers.

7.1 The three configuration files

File	Role
`CLAUDE.md` (repo root)	Long-form, *checked-in* operating instructions Claude reads at every session start. Contains the §6 issue-tracking rules, the §5 testing policy, the §8 doc checklist, and the brand/content invariants.
`C:\Repos\CLAUDE.md` (parent dir)	Cross-project policy: plan-mode default, subagent strategy, self-improvement loop, demand-elegance rule. Inherited by every repo under `C:\Repos\`.

`.claude/settings.local.json`	Per-machine permissions allowlist (tools, bash patterns, MCP servers). Not committed to the public repo.

7.2 Slash commands / skills

Claude Code skills live under `.claude/skills/`. RAV currently ships **one project skill**:

- **`/sdlc`** (`.claude/skills/sdlc/SKILL.md`) — the full development lifecycle: priority review → issue pickup → brand check → implementation → testing → deployment → issue close + doc updates. Invoked as `/sdlc`, `/sdlc status`, or `/sdlc <issue-number>`.

Other slash commands referenced in the harness (`/loop`, `/schedule`, `/security-review`, `/review`, `/init`, `/simplify`, `/fewer-permission-prompts`, `/update-config`, `/keybindings-help`, `/claude-api`) are **Claude Code built-in** / **global skills**, not project-specific. They're available everywhere Claude Code runs.

7.3 MCP servers in use

MCP (Model Context Protocol) servers are how Claude reaches outside its sandbox. Three are active for this project:

MCP server	What it unlocks	Representative tools
GitHub	Read/write issues, PRs, comments, files, branches without shelling out to `gh`	`mcp__github__list_issues`, `issue_read`, `issue_write`, `add_issue_comment`, `update_pull_request`, `search_issues`, `sub_issue_write`, `get_file_contents`
Sentry	Pull live error telemetry directly into the conversation when triaging bugs	`mcp__sentry__search_issues`, `search_events`, `get_sentry_resource`
Google Drive	Read brand/marketing assets, QA scenario spreadsheets, exports	`mcp__claude_ai_Google_Drive__list_recent_files`, `read_file_content`, `download_file_content`

There is **no `.mcp.json` at repo root** — MCP server registration happens via the Claude Code app, and the project-level allowlist in `.claude/settings.local.json` simply enables the specific tools.

Considered & not yet added:

- *Supabase MCP* — schema introspection and SQL via MCP would replace several manual `psql` flows. Not added because the existing `supabase` CLI plus generated TypeScript types covers most of the need.

- *Vercel MCP* — deploy logs in-conversation. Vercel's GitHub status checks are sufficient today.
- *Linear / Notion MCP* — n/a; GitHub Issues is the system of record.

7.4 Two-tier AI workflow (close to spec-driven, not formally)

In practice, two distinct Claude surfaces handle different jobs:

1. **Strategy tier — Claude.ai (web).** Long, exploratory conversations to work out design, architecture, and trade-offs. Output is a written brief or set of instructions — what to build, how it should behave, what was considered and rejected.
2. **Execution tier — Claude Code (CLI).** The brief is brought into the repo (often as edits to `CLAUDE.md`, an issue body, or session prompt). Claude Code carries it through branches, code, tests, doc updates, and the PR.

This is **adjacent to spec-driven development**, but not formally so. In true SDD the spec is a versioned, tracked artifact that lives in the repo, drives tests, and is updated alongside the code. Here, the brief is consumed once: its substance survives in the resulting `CLAUDE.md` edits, GitHub issue body, or PR description, but the architect-level reasoning is not preserved as a long-lived artifact.

It still earns its keep — the split keeps the executor focused (small context, fast loop) and the architect ambitious (big context, no execution pressure). But the upgrade to true SDD is real and inexpensive; flagged in §16.

7.5 The "Boardroom" pattern

The **Boardroom** is not a slash command or piece of automation — it's a **documentation pattern** for capturing structural / strategic decisions that don't belong in a GitHub Issue. Each boardroom session lives under `docs/boardroom/sessions/<YYYY-MM-DD>-<topic>/` and contains:

- The debate transcript (`.md`)
- An interactive HTML view (`.html`)
- A printable export (`.pdf`)
- A `summary.md/html/pdf` triplet
- A `manifest.md` describing the participants and outcome

Existing boardroom sessions cover decisions like *Beta launch before LLC?*, *Delaware vs Florida incorporation*, *Holding-company structure*, and *Incorporation service choice*. The pattern is reusable for any project where high-stakes, multi-perspective decisions need a permanent record separable from code history.

---

8. Session continuity — the docs system

The hardest problem with AI-assisted development is **memory across sessions**. Solved here with a

four-tier doc system, all under `docs/`, all enforced by automation.

8.1 Tier-1 — read every session, audited every PR

File	Purpose
`docs/PROJECT-HUB.md`	Central command center. Holds the **Key Decisions Log** (DEC-### entries), the rolling **Session Handoff** (current + last ~5 sessions), and the **Platform Status** block (test count, migration count, edge functions, dev/main sync state).
`docs/PRIORITY-ROADMAP.md`	Tiered list of what to work on next. Updated with a revision-history entry every prioritization session.
`docs/testing/TESTING-STATUS.md`	Live test inventory (currently 1402 tests across 150 files; 201 are `@p0` critical path; 30 manual QA scenarios).
`docs/LAUNCH-READINESS.md`	Pre-flight checklist for disabling Staff-Only mode in production.

A custom script — **`scripts/docs-sync-check.ts`** (`npm run docs:sync-check`) — programmatically verifies that all four are within tolerance:

- PROJECT-HUB Session Handoff matches latest session in git
- PRIORITY-ROADMAP "as of Session NN" within ±1 of git
- TESTING-STATUS test count matches `test-results/junit.xml` within ±5 %
- LAUNCH-READINESS frontmatter `change_type` within 3 sessions of git

The same script runs in CI via `.github/workflows/docs-audit.yml` — **stale Tier-1 docs fail PRs**.

8.2 Tier-2 — durable context, read on demand

File	Purpose
`docs/COMPLETED-PHASES.md`	Detailed archive of finished work. Sessions older than ~5 are moved here from PROJECT-HUB to keep the hub lean.
`docs/ARCHITECTURE.md`	40-section developer guide: tech stack, routing, RBAC, schema, edge functions, email, pricing, realtime, APIs, design system. Auto-references the flow manifests in `src/flows/`.
`docs/PLATFORM-INVENTORY.md`	One-pager mental model of features + infra + tooling + governance. Read before designing anything cross-cutting.
`docs/DECISIONS.md`	Archived `DEC-###` entries promoted out of PROJECT-HUB once stable.

8.3 Tier-3 — domain locks

File	Purpose

`docs/brand-assets/BRAND-LOCK.md`	Source-of-truth for brand vocabulary (the noun lock: *Listing / Wish / Offer*), positioning hierarchy, email tone. **If anything else conflicts with this, this wins.**
`docs/RAV-PRICING-TAXES-ACCOUNTING.md`	Pricing math, take-rate, fee model.
`docs/SENTRY-GUIDE.md`	How error reporting is wired and what to do when an issue triggers.
`src/flows/*.ts`	Declarative **flow manifests** (owner / traveler / admin lifecycles). The `/architecture` page in-app *auto-renders Mermaid diagrams* from these — no hand-authored Mermaid.

8.4 Tier-4 — support material

`docs/api/`, `docs/guides/`, `docs/marketing/`, `docs/payments/`, `docs/strategy/`, `docs/incorporation/`, `docs/support/`, `docs/exports/`. Read when relevant, not on every session.

8.5 Frontmatter convention

Every `docs/**/*.md` file carries:

```
---
last_updated: "2026-03-13T14:30:00"   # auto-stamped by docs-stamp.sh
change_ref: "abc1234"                  # auto-stamped — short SHA
change_type: "session-40"              # manual — session #, PR #, or "manual-edit"
status: "active"                       # active | archived | draft
---
```

`archived` docs are skipped by the staleness check, so old material stays in-tree without raising false alarms.

8.6 The end-of-session checklist

Codified in `CLAUDE.md` and run after every merged PR. The 13-step checklist covers:

1. Close the GitHub issue with the `Completed:` / `Technical:` summary
2. Update `PRIORITY-ROADMAP.md`
3. Update `PROJECT-HUB.md` (handoff entry, platform-status numbers, decisions log)
4. Update `testing/TESTING-STATUS.md` if tests changed
5. Update `LAUNCH-READINESS.md` if a gate moved
6. Update `ARCHITECTURE.md` + flow manifests if structure changed
7. Update `UserGuide.tsx` + FAQ if user-visible behavior changed
8. Update `testing/QA-PLAYBOOK.md` if a workflow changed
9. Update `brand-assets/BRAND-LOCK.md` if terminology changed
10. Move aged session entries into `COMPLETED-PHASES.md`
11. Open follow-up issues for anything discovered mid-session

12. Save anything novel to **auto-memory** (Claude's per-user persistent memory — feedback / project / user / reference notes)
13. Run `npm run docs:sync-check` as the safety net

The *checklist* tells you what to update; **`docs:sync-check`** tells you whether you actually did. CI is the backstop.

8.7 Source-to-doc mapping

`scripts/source-doc-map.json` maps source paths to docs. When a source file changes without its mapped doc, the audit warns. New mappings are added when feature docs are created.

---

9. Testing strategy

9.1 Layers

Layer	Tool	Where	Notes
Unit	**Vitest 3**	`src/lib/*.test.ts`	Pure functions, especially financial math (commission, fees, refunds)
Hook integration	**Vitest**	`src/hooks/*.test.ts`, `src/hooks/**/__tests__/*`	Mock Supabase via `src/test/helpers/supabase-mock.ts`
Context integration	**Vitest**	`src/contexts/*.test.tsx`	Auth, role, and feature-flag contexts
E2E smoke	**Playwright 1.58**	`e2e/smoke/`	Top critical paths only — kept small for speed
Visual regression	**Percy** + `@percy/playwright`	`e2e/visual/`	Snapshots gated to key pages
Performance	**Lighthouse CI 0.15**	invoked from `ci.yml`	Catches perf regressions on production-like builds

9.2 Test fixtures & helpers

```
src/test/fixtures/users.ts        — mockUser, mockSession, mockProfile, mockAuthContext
src/test/fixtures/listings.ts     — listing fixtures
src/test/fixtures/memberships.ts  — tier fixtures
src/test/helpers/render.tsx       — createHookWrapper, renderWithProviders
src/test/helpers/supabase-mock.ts — createSupabaseMock, emptyResponse, errorResponse
```

9.3 Tests-with-features policy (mandatory)

Every feature or bug fix lands with tests **in the same PR**, not as follow-up. At minimum:

- Any new `src/lib/` function gets a unit test
- Any new hook gets an integration test
- Any new context method extends the context test
- Financial math gets edge-case coverage
- Happy path + at least one error case for any new code path

9.4 Coverage thresholds (enforced in CI)

- Statements 25 %, Branches 25 %, Functions 30 %, Lines 25 %.

Deliberately moderate — high coverage on critical paths (financial math, auth) is more useful than chasing 80 % everywhere.

9.5 QA layer

- **Manual QA scenarios** live in a Google Drive spreadsheet (numbered `S-01`, `S-02`, …, `S-30`); narrative in `docs/testing/QA-PLAYBOOK.md`.
- **`@p0` tag** in unit tests marks the 201 tests that gate launch.
- **Qase.io was evaluated** as a TestRail-style scenario manager. Decided against — Google Drive + the playbook covers the need without another subscription.

9.6 Considered & rejected

- *Cypress* — Playwright won (better debugging, multi-browser, faster).
- *Jest* — Vitest is native to the Vite build, faster startup, drop-in `expect`.
- *Storybook + Chromatic* — Percy snapshots cover the visual-regression need without a parallel component-catalog system to maintain.

---

10. Observability

Concern	Tool
Frontend errors	**Sentry** — wired through `@sentry/react`; setup notes in `docs/SENTRY-GUIDE.md`
Triage in-conversation	**Sentry MCP server** (§7.3) lets Claude pull live issue + event detail without tab-switching
Email/digest reporting	**Resend** — GitHub Actions sends the daily status email and per-issue notifications

Vercel analytics	Default Vercel analytics on the production project
Lighthouse	LHCi reports archived on every CI run

Considered & rejected:

- *Datadog / New Relic* — too heavy for the current scale; Sentry covers the actionable layer.
- *PostHog / Amplitude* — product analytics deferred until post-launch.

---

11. Dependency management

Currently: *manual* `npm` updates. There is no `dependabot.yml`, no Renovate config, no scheduled `npm audit` workflow.

Considered:

- **Dependabot** — would be the obvious GitHub-native pick. Deferred because the repo is fast-moving and a wave of PRs would crowd out feature work. Will be enabled with a *weekly grouped* schedule once launch is locked.
- **Renovate** — more configurable than Dependabot, especially for grouping and auto-merge of safe ranges. Likely the long-term choice; the trade is one more YAML to own.
- **Snyk** / **Mend** — security-first scanners. Useful, but `npm audit` plus Sentry for runtime issues is sufficient pre-launch.

Action item for new projects: turn on grouped weekly Renovate (or Dependabot) from day one — paying the noise cost early is cheaper than catching up under deadline.

---

12. Environment & secrets

Where	What
`.env.local` (gitignored)	`VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY` pointing at **DEV**
`.env.example`	Template (Supabase URL/key, Twilio placeholders, SMS test-mode flag)
Vercel project env	PROD Supabase keys, Stripe live keys, Resend API key, VAPI keys
Supabase Function env	Secrets exposed only to edge functions

No `.nvmrc` is committed — Node 20 is documented in `CLAUDE.md` and used by CI. Worth adding `.nvmrc` for new projects to make local switches automatic.

13. Tooling decisions log — at-a-glance

Concern	Using now	Considered & dropped (or deferred)	Why
Hosting	Vercel (GitHub app)	Netlify, Cloudflare Pages	Best-in-class DX with the Next/React + edge-function model; no token to manage
Backend	Supabase	Firebase, custom Express + Postgres	Postgres + RLS + edge functions + realtime in one platform; matches the auth/RBAC needs
Payments	Stripe + PaySafe escrow	Stripe-only	Need long-hold escrow that Stripe Connect can't cleanly model
Voice	VAPI (Deepgram + GPT-4o-mini + ElevenLabs)	Twilio Voice + custom stack	VAPI bundles STT/LLM/TTS; faster to ship
Mobile	Capacitor (post-PWA) — DEC-011	React Native	Reuse the React web codebase; PWA validates demand first
Unit/integration tests	Vitest	Jest	Native to Vite, faster
E2E	Playwright	Cypress	Better cross-browser + debugging
Visual	Percy	Chromatic	Already in CI; no Storybook tax
Perf	Lighthouse CI	WebPageTest	Free, GitHub-native
Lint	ESLint	+ Prettier	Single tool; ESLint formatting rules suffice
Commit lint	(none)	commitlint + Husky	Convention documented in CLAUDE.md; manual discipline at this team size
Pre-commit	Husky + lint-staged + docs-stamp	Pre-push, Lefthook	Husky + lint-staged is the deepest-supported combo
Issue tracking	GitHub Issues + Milestones	Linear, Jira, Notion	Co-located with code; first-class GitHub MCP
AI assistant	Claude Code (Opus 4.7)	Cursor, Copilot Workspace	Best skills/MCP/hooks story; the SDLC skill ties it all together

MCP servers	GitHub, Sentry, Google Drive	Supabase MCP, Vercel MCP, Linear MCP	Add when the manual flow becomes the bottleneck
Error monitoring	Sentry	Datadog, New Relic	Right-sized for current scale
Email	Resend	SendGrid, Postmark	Simpler API; good DX from Vercel
Test mgmt	Google Drive + QA-PLAYBOOK.md	Qase.io, TestRail	Avoid a subscription until the team grows
Dependency updates	Manual `npm`	Dependabot, Renovate, Snyk	Enable Renovate post-launch
Docs portal	Markdown in repo + custom audit	Docusaurus, Mintlify, Notion	Markdown + frontmatter + CI audit gives 95 % of the value

---

14. Cross-project starter checklist

When you spin up a new application and want to start from this model:

Week 1 — repo & deploy

- [ ] Create GitHub repo with `main` protected (PR + 1 review + CI required)
- [ ] Create `dev` branch; set as default working branch
- [ ] Connect Vercel GitHub app — preview on every branch, prod on `main`
- [ ] Add `vercel.json` with security headers
- [ ] Provision two Supabase projects: DEV (used locally) and PROD (Vercel prod env)
- [ ] Add `.env.example`, gitignore `.env.local`, document key names

Week 1 — Claude harness

- [ ] Drop a `CLAUDE.md` at repo root with: project facts, branching rules, testing rules, end-of-session checklist
- [ ] Add `.claude/settings.local.json` with the MCP allowlist you want (start with GitHub MCP)
- [ ] Create one `.claude/skills/sdlc/SKILL.md` for the dev-lifecycle slash command

Week 2 — quality gates

- [ ] `npm i -D vitest @vitest/coverage-v8 @testing-library/react jsdom`
- [ ] `npm i -D playwright @percy/playwright @percy/cli`
- [ ] `npm i -D husky lint-staged eslint typescript-eslint`
- [ ] `npx husky init` → add the `pre-commit` hook running `lint-staged`
- [ ] Add `ci.yml`: lint → typecheck → unit → e2e → visual → lighthouse

- [ ] Wire Sentry (`@sentry/react`) and add the Sentry MCP server

Week 2 — docs scaffolding

- [ ] Create `docs/PROJECT-HUB.md`, `docs/PRIORITY-ROADMAP.md`, `docs/ARCHITECTURE.md`, `docs/LAUNCH-READINESS.md`, `docs/testing/TESTING-STATUS.md`
- [ ] Adopt the frontmatter convention (`last_updated`, `change_ref`, `change_type`, `status`)
- [ ] Port `scripts/docs-stamp.sh`, `scripts/docs-audit.ts`, `scripts/docs-sync-check.ts` and the matching `npm run docs:*` scripts
- [ ] Add `docs-audit.yml` workflow

Week 3 — process

- [ ] Create initial Milestones (e.g., `MVP`, `Launch Readiness`, `Phase 2`)
- [ ] Create the label set (§6.1)
- [ ] Add the `daily-summary.yml` and `issue-notifications.yml` workflows pointing at Resend
- [ ] Decide: enable Renovate now (recommended) or defer

When the time comes

- [ ] Open a `docs/boardroom/sessions/YYYY-MM-DD-<topic>/` folder for any structural / legal / strategic decision that doesn't fit a GitHub issue

---

15. What this model is *not* good at

Honest about the trade-offs:

- **Multi-engineer teams.** No CODEOWNERS, no PR templates, no commitlint, no automated reviewer assignment. Add those before adding the third engineer.
- **Compliance-heavy domains.** No SOC 2-grade audit trail beyond git + GitHub Issues. Boardroom docs help but aren't a substitute.
- **Mobile-first products.** Capacitor is fine for an extension; for a native-first product, this model would lean too web-shaped.
- **High-velocity dependency churn.** Manual `npm` updates are pleasant *until* a CVE drops. Renovate before launch.

---

16. Possible enhancements

Items the model would benefit from — none blocking today, all worth the small lift before the team or the surface area grows.

- **Promote the two-tier AI workflow to true spec-driven development (§7.4).** Create `docs/`

`specs/` (or `docs/architect-briefs/`), drop the Claude.ai brief there before any major change, link it from the resulting GitHub issue and PR. Cost: one new convention plus a brief template. Benefit: the original architect-level reasoning survives long after the code lands, so future sessions can re-derive *why* a thing was built that way without spelunking through chat history.

- **Enable Renovate on a grouped-weekly schedule (§11).** Pre-launch noise is the only reason it's deferred. Turn it on with grouping rules so security and dev-dep updates batch together instead of one-PR-per-bump.
- **Add CODEOWNERS, a PR template, and commitlint (§15).** Trivial day-of-onboarding tax for the third engineer; painful retrofit afterwards.
- **Add a Supabase MCP server (§7.3).** When manual `psql` / `supabase` CLI flows start eating session time, a Supabase MCP would let Claude introspect schema and run safe queries inline.
- **Add `.nvmrc` and a `pre-push` typecheck (§5, §12).** Cheap insurance against "works on my machine" once collaborators land.

17. Pointers

- Repo: https://github.com/rent-a-vacation/rav-website
- Production: https://rent-a-vacation.com
- Vercel project: https://rentavacation.vercel.app
- Supabase PROD: `xzfllqndrlmhclqfybew`
- Supabase DEV: `oukbxqnlxnkainnligfz`
- Daily status digest: emailed at 04:30 UTC by `daily-summary.yml`

This document is itself part of Tier-2 — durable context. Update it when the operating model changes, not when the project does.